



AI 서비스를 위한 SW 기초 Full-stack Engineering

- 스마트 데이터 이해 및 활용
- day03_인덱스 설계 및 최적화
- 2026.07

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

데이터베이스 성능 향상을 위한 인덱스 설계 및 최적화

100만 건 orders 테이블 예시로 배우는 B-Tree 인덱스 및 실무 최적화 전략

1. 인덱스(Index)의 핵심 개념과 동작 원리

인덱스가 없을 때 (Seq Scan)

100만 개의 전체 데이터 블록을 처음부터 끝까지 순차적으로 탐색

데이터량이 증가할수록 선형적으로 탐색 시간이 증가 ($O(N)$)

책의 첫 페이지부터 원하는 단어가 나올 때까지 한 장씩 다 넘겨보는 것

인덱스가 있을 때 (Index Scan)

B-Tree 구조를 통해 원하는 행의 물리적 위치(TID)로 즉시 점프

데이터량이 폭발해도 탐색 속도가 일정 수준으로 유지 ($O(\log N)$)

책 맨 뒤의 '색인'을 보고 몇 페이지에 특정 단어가 있는지 바로 찾아가는 것

2. 복합 인덱스와 '선두 컬럼의 법칙'

```
-- 유저 ID와 주문 생성일을 결합한 복합 인덱스 생성 예시  
CREATE INDEX idx_orders_user_created ON orders(user_id, created_at DESC);
```

쿼리 조건: WHERE user_id = 12345 | 성능: **o (활용 가능)**

설명: 선두 컬럼(user_id)이 조건절에 포함되어 빠르게 탐색

쿼리 조건: WHERE user_id = 12345 AND created_at >= '2026-01-01' | 성능: **o (최적 성능)**

설명: 두 컬럼 모두 인덱스 범위 검색에 정밀하게 활용됨

쿼리 조건: WHERE created_at >= '2026-01-01' | 성능: **x (효과 없음/제한)**

설명: 선두 컬럼이 빠져 인덱스 스캔 불가 (Seq Scan 유도)

💡 **핵심 원칙:** 전화번호부에서 '성' 없이 '이름'으로 찾을 수 없듯,
복합 인덱스는 무조건 선두 컬럼부터 조건절에 사용해야 합니다.

3. 주문 테이블(orders) 실전 인덱스 설계 대상

01. 자주 조회되는 외래키 (FK)

- 조건절: WHERE user_id = ?
- Recommended Index: idx_orders_user_id

유저별 주문 목록을 조회할 때 필수적인 가장 기본 인덱스

02. 기간 / 날짜 범위 검색

- 조건절: WHERE created_at BETWEEN ...
- Recommended Index: idx_orders_created_at

일별, 월별 매출 집계 및 거래 내역 기간 조회 시 활용

03. 정렬 연산 오버헤드 방지

- 조건절: ORDER BY created_at DESC
- Recommended Index: idx_orders_created_at

인덱스는 이미 정렬되어 있으므로 In-Memory 정렬 비용 완전 제거

04. 복합 조건 조회 (유저 + 상태)

- 조건절: WHERE user_id = ? AND order_status = ?
- Recommended Index: idx_orders_user_status

특정 유저의 '배송중' 또는 '결제완료' 상태 주문만 추출 시 최적

4. 인덱스의 트레이드오프 (Trade-off)

주문 1건 추가(INSERT) 시 DB 동작

1. orders 원본 테이블에 레코드 추가
2. idx_orders_user_id B-Tree 재정렬 및 추가
3. idx_orders_created_at B-Tree 재정렬 및 추가
4. idx_orders_user_status B-Tree 재정렬 및 추가
- ...

인덱스 과다 생성 시 주요 부작용



INSERT, UPDATE, DELETE 발생 시 모든 인덱스 트리를 매번 재정렬해야 함



배보다 배꼽이 더 커져 인덱스가 원본 테이블 용량을 초과하기도 함



선택지가 너무 많아져 DB가 실행 계획을 찾는 시간 자체도 증가함

5. 핵심 요약 (Summary)

실무 인덱스 설계 3대 가이드라인

적절한 인덱스는 대용량 환경에서 Seq Scan을 Index Scan으로 전환해 수백 배의 속도 향상을 제공합니다.

복합 인덱스 작성 시 쿼리의 WHERE 절 조건에서 가장 자주, 정확히(EQUAL) 사용되는 컬럼을 맨 앞에 배치하세요.

불필요한 인덱스는 즉시 정리하고, 시스템의 조회 대비 쓰기 비율을 고려하여 최소한의 인덱스로 최대의 효과를 내야 합니다.

💡 "인덱스는 SELECT 속도를 높이지만, INSERT·UPDATE·DELETE 비용과 저장 공간을 담보로 사용하는 양날의 검이다."



• 수고하셨습니다.

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.